

Multi-Task and Hierarchical RL

CS 185/285

Instructor: Sergey Levine
UC Berkeley



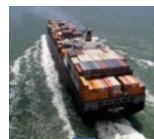
Part 1:

What is multi-task RL and why should we care?

Generalist models



segmentation
model

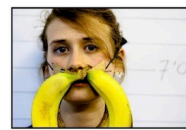


classification
model



captioning
model

A group of people
shopping at an
outdoor market.



What is the mustache
made of?

visual QA
model

bananas

"Horrible services. The room
was dirty and unpleasant.
Not worth the money."

sentiment
model



Natural Language Processing

Natural language processing (NLP) is a subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data. The result is a computer capable of "understanding" the contents of documents, including the contextual nuances of the language within them. The technology can then accurately extract information and insights contained in the documents as well as categorize and organize the documents themselves.

summarization
model

Natural Language Processing

Natural language processing (NLP) is a subfield of linguistics, computer science, and artificial intelligence concerned with the interactions between computers and human language, in particular how to program computers to process and analyze large amounts of natural language data.



general-purpose
foundation model



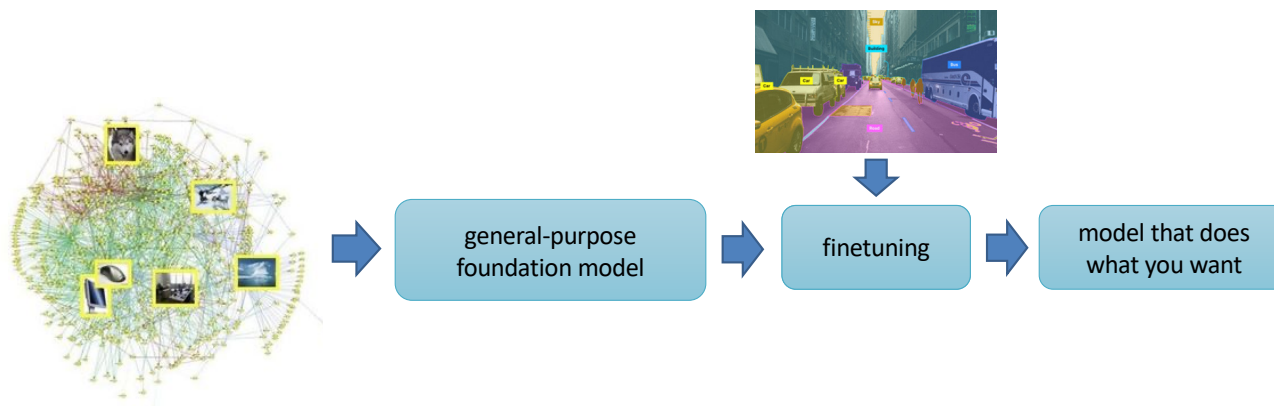
finetuning



model that does
what you want

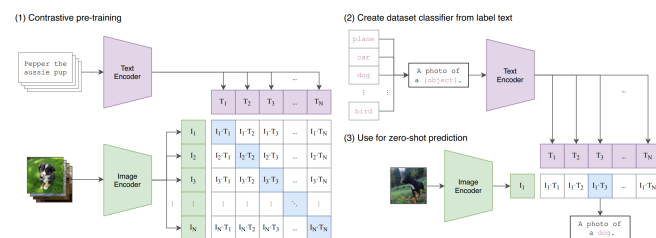


Generalist models

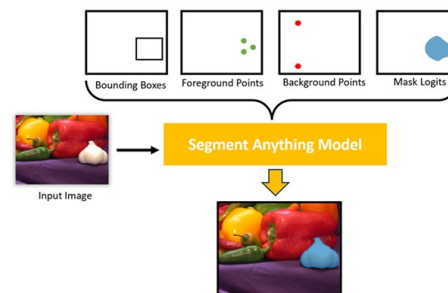


Generalist models are everywhere!

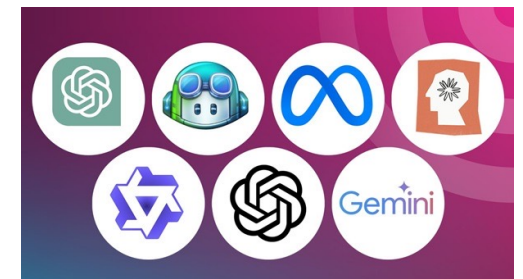
Key idea is **multi-task learning**: by training on many tasks at once, generalist models require much less data of any single task



CLIP (image labeling)



SAM (computer vision)



LLMs

Multi-task RL

Regular RL:

$$\theta \leftarrow \arg \max_{\theta} E_{\tau \sim \pi_{\theta}(\tau)} [r(\tau)]$$

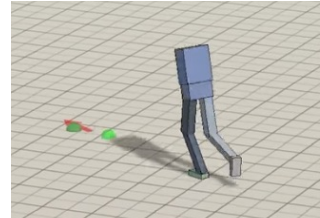
Multi-task RL:

$$\theta \leftarrow \arg \max_{\theta} E_{\omega \sim p(\omega)} [E_{\tau \sim \pi_{\theta}(\tau|\omega)} [r_{\omega}(\tau)]]$$

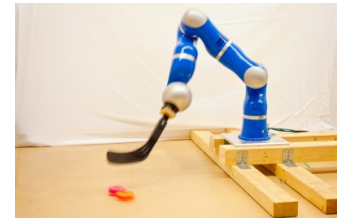
“task context”

$$\sum_{t=1}^H r_{\omega}(\mathbf{s}_t, \mathbf{a}_t)$$

$$\pi_{\theta}(\tau|\omega) = p(\mathbf{s}_1) \prod_{t=1}^H \pi_{\theta}(\mathbf{a}_t|\mathbf{s}_t, \omega) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$



ω : walking direction



ω : where to hit puck

What is task context?

language command

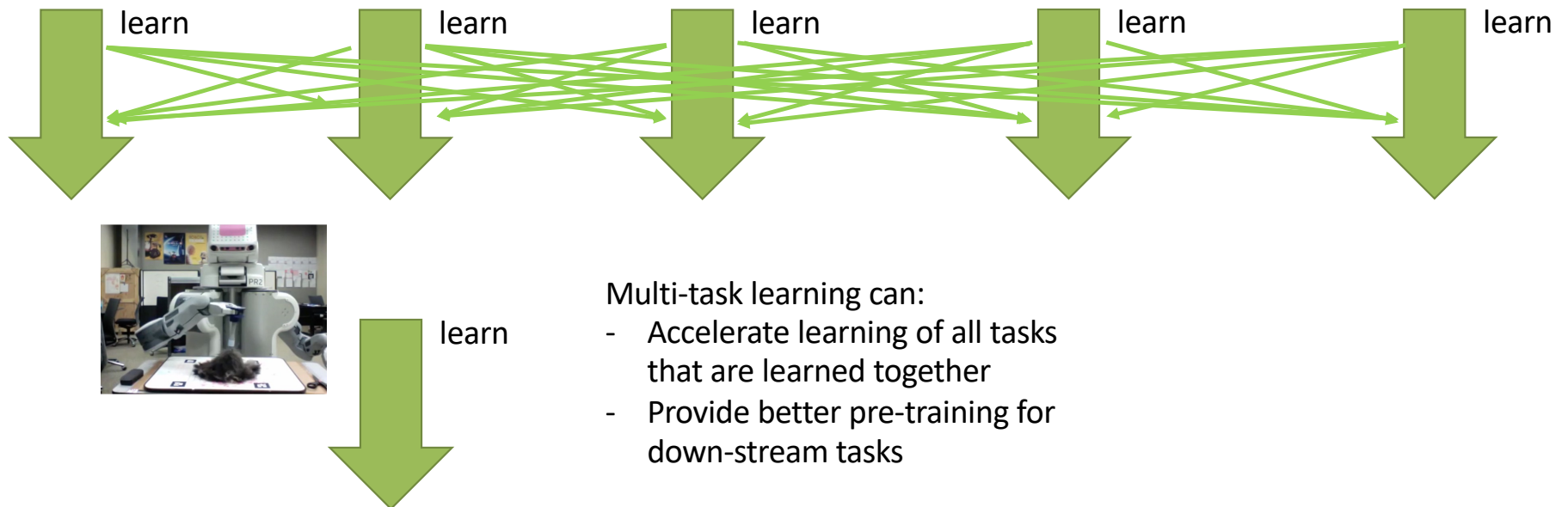
goal

which game the agent is playing

which location an autonomous car should drive to

Did we see this before?

Multi-task RL for *efficiency*



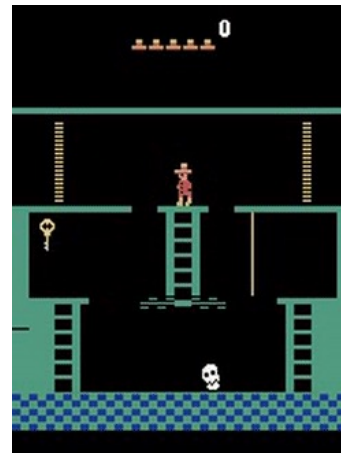
Multi-task RL for *transfer*

this is easy (mostly)



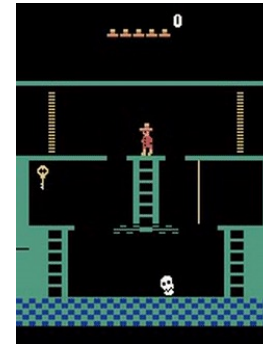
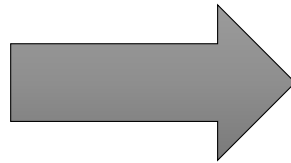
Why?

this is impossible



- Getting key = reward
- Opening door = reward
- Getting killed by skull = bad
- We know what to do because we **understand** what these sprites mean!
- Key: we know it opens doors!
- Ladders: we know we can climb them!
- Skull: we don't know what it does, but we know it can't be good!
- **Prior understanding of problem structure can help us solve complex tasks quickly!**

Multi-task RL for *transfer*



- If we've solved prior tasks, we might acquire useful knowledge for solving a new task
- How is the knowledge stored?
 - Q-function: tells us which actions or states are good
 - Policy: tells us which actions are potentially useful
 - some actions are never useful!
 - Models: what are the laws of physics that govern the world?
 - Features/hidden states: provide us with a good representation

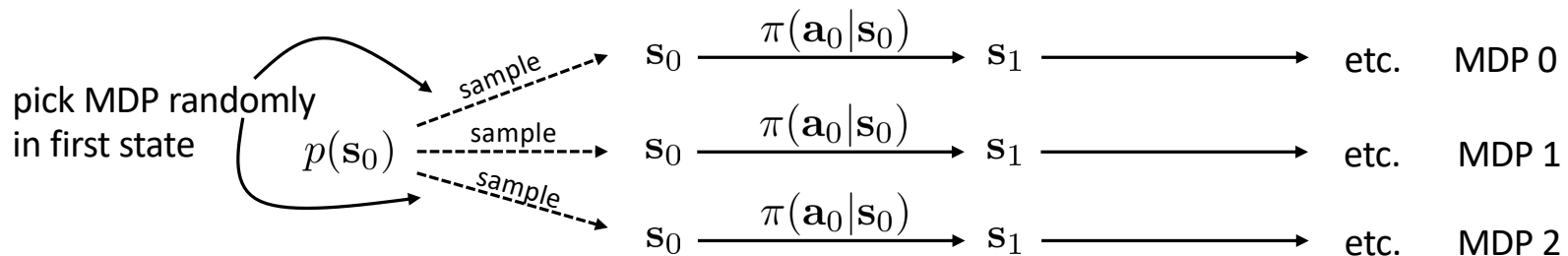
Part 2:

The basics of multi-task RL

Is multi-task RL *different* from regular RL?

No

Multi-task RL corresponds to single-task RL in a **joint MDP**



just add the context ω to the state!

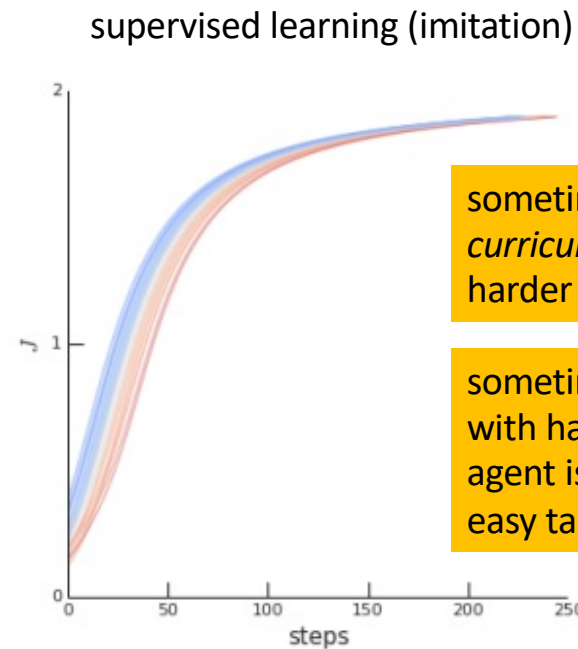
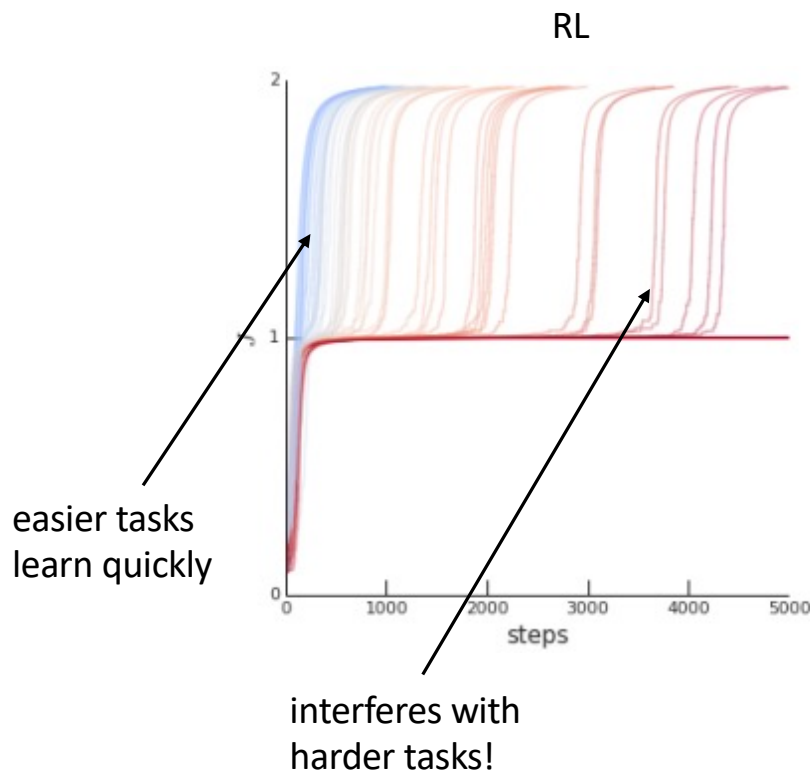
From regular RL to multi-task RL

1. input ω into $\pi_{\theta}(\mathbf{a}|\mathbf{s}, \omega)$
2. append ω to the state $\mathbf{s}$
3. for each trajectory τ , sample $\omega \sim p(\omega)$ and use r_{ω}

How?

You already did this in HW4!

Why is multi-task RL difficult?



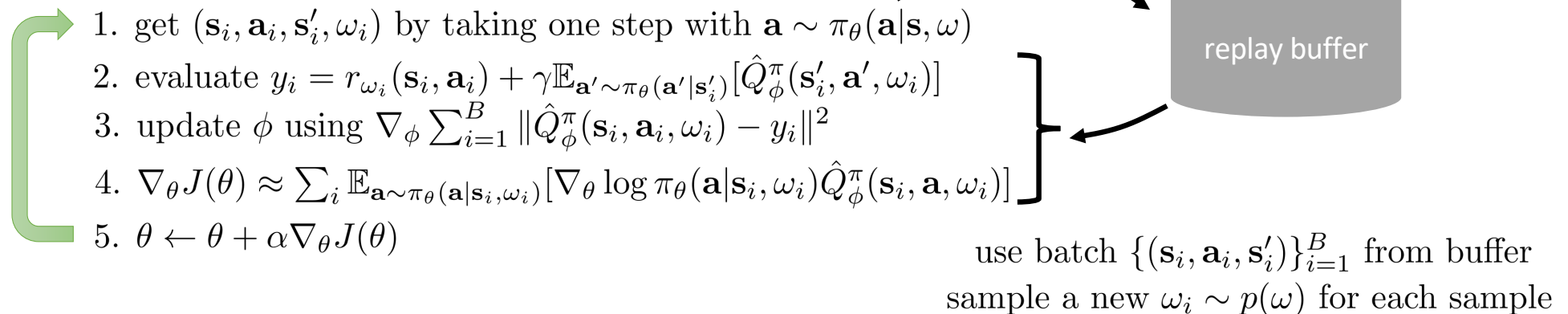
Schaul et al. Ray Interference: a Source of Plateaus in Deep Reinforcement Learning.

What can we do to make it less difficult?

Curriculum: easy tasks can help us learn something useful, which might help us later solve harder tasks

Relabeling: we can reuse past experience with different contexts!

off-policy actor-critic algorithm:

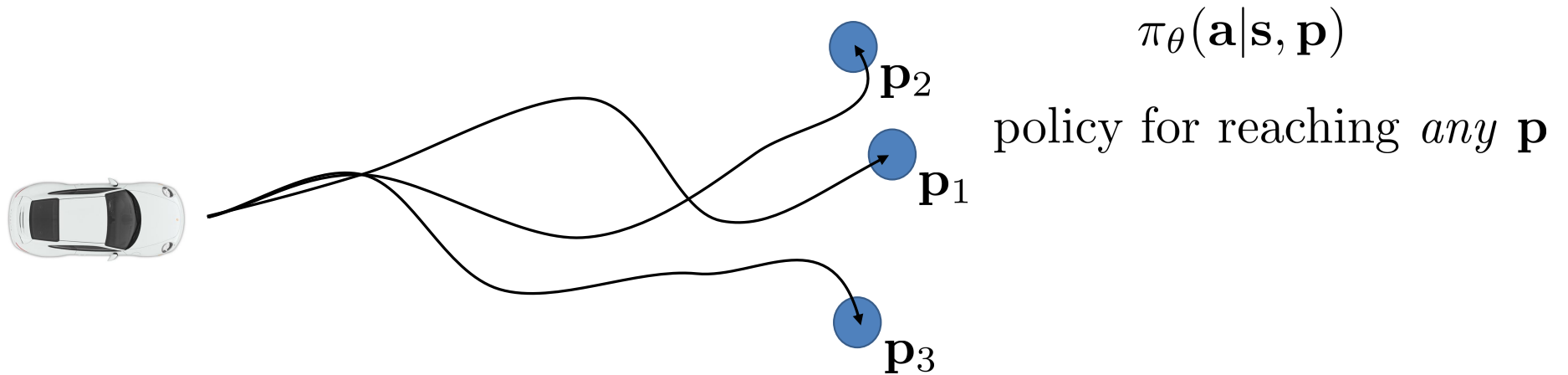


“data augmentation” trick that lets us squeeze more out of the same data

Part 3:

Goal-conditioned RL

A special kind of multi-task RL



In general:

$$\pi_{\theta}(\mathbf{a}|\mathbf{s}, \mathbf{g})$$

goal state

Goal-conditioned policies

$$\pi_{\theta}(\mathbf{a}|\mathbf{s}, \mathbf{g})$$



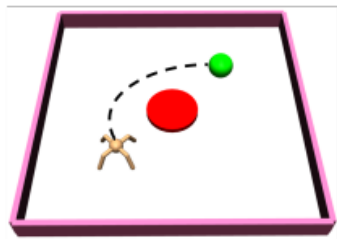
another state

$$r(\mathbf{s}, \mathbf{a}, \mathbf{g}) = \delta(\mathbf{s} = \mathbf{g})$$

$$r(\mathbf{s}, \mathbf{a}, \mathbf{g}) = \delta(\|\mathbf{s} - \mathbf{g}\| \leq \varepsilon)$$

$$r(\mathbf{s}, \mathbf{a}, \mathbf{g}) = -\delta(\mathbf{s} \neq \mathbf{g})$$

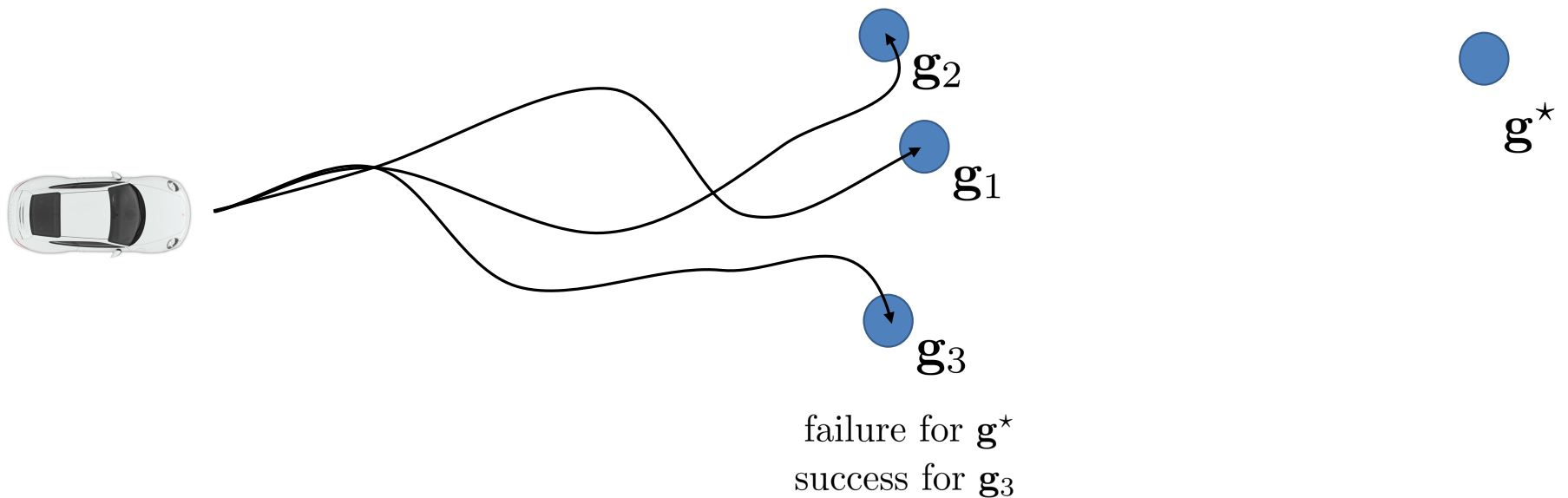
- Convenient: no need to manually define rewards for each task
- Can transfer in **zero shot** to a new task if it's another goal!
- Often hard to train in practice (see references)
- Not all tasks are goal reaching tasks!



A few relevant papers:

- Kaelbling. **Learning to achieve goals.**
- Schaul et al. **Universal value function approximators.**
- Andrychowicz et al. **Hindsight experience replay.**
- Eysenbach et al. **C-learning: Learning to achieve goals via recursive classification.**

Relabeling goals



Key idea: trajectories that **fail** for the goal that was commanded still **succeed** for some other goal!

This can make it much easier to learn, because we get much denser rewards!

Example goal-conditioned RL algorithm

off-policy actor-critic algorithm:

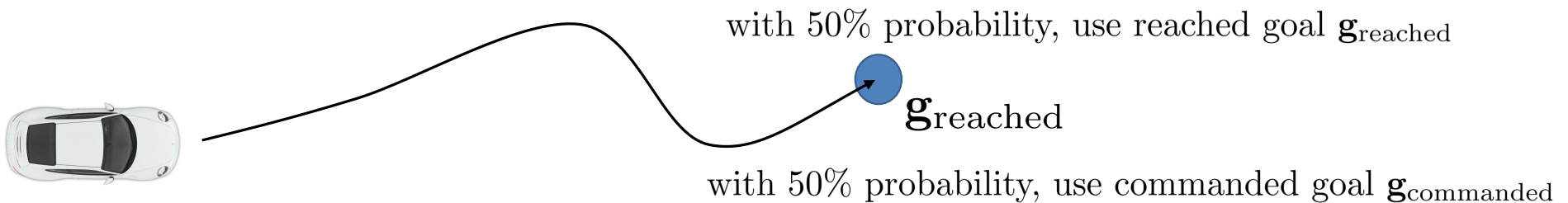
1. get $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, \mathbf{g}_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}, \mathbf{g})$
2. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i, \mathbf{g}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{s}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}', \mathbf{g}_i)]$
3. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i, \mathbf{g}_i) - y_i\|^2$
4. $\nabla_\theta J(\theta) \approx \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{s}_i, \mathbf{g}_i)} [\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{s}_i, \mathbf{g}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}, \mathbf{g}_i)]$
5. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

store $\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, \mathbf{g}_i$

replay buffer

use batch $\{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i)\}_{i=1}^B$ from buffer
sample a new $\mathbf{g}_i$ for each sample

Question: why do we need to use an off-policy algorithm here?



There are many different “relabeling strategies” we can use here!

A bit of geometry

episode “ends” when $\mathbf{s} = \mathbf{g}$

$$V(\mathbf{s}, \mathbf{g}) = -\delta(\mathbf{s} \neq \mathbf{g}) + E_{\mathbf{s}'}[V(\mathbf{s}', \mathbf{g})]\delta(\mathbf{s} \neq \mathbf{g})$$

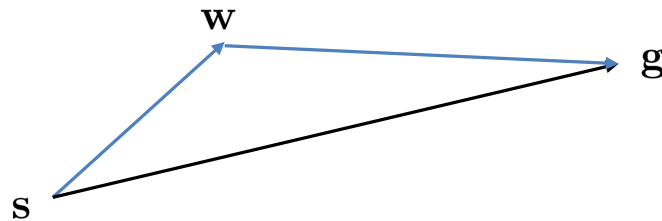
$$r(\mathbf{s}, \mathbf{a}, \mathbf{g}) = -\delta(\mathbf{s} \neq \mathbf{g}) \quad \gamma = 1$$

$$= \begin{cases} -1 + E_{\mathbf{s}'}[V(\mathbf{s}', \mathbf{g})] & \text{if } \mathbf{s} \neq \mathbf{g} \\ 0 & \text{if } \mathbf{s} = \mathbf{g} \end{cases}$$

“number of time steps from $\mathbf{s}$ to $\mathbf{g}$ ”

this behaves a bit like a distance function!

$$V(\mathbf{s}, \mathbf{g}) \leq V(\mathbf{s}, \mathbf{w}) + V(\mathbf{w}, \mathbf{g})$$



This can be used for **planning** with goal conditioned value functions!

Nasiriany et al. **Planning with Goal-Conditioned Policies.**

This can also be used as an **auxiliary loss function**!

Kaelbling. **Learning to achieve goals.**

Intermission

Part 4:

Successor representations

Claim: goal-conditioned RL is a lot like a model

$$V(\mathbf{s}, \mathbf{g}) = \begin{cases} -1 + E_{s'}[V(s', \mathbf{g})] & \text{if } \mathbf{s} \neq \mathbf{g} \\ 0 & \text{if } \mathbf{s} = \mathbf{g} \end{cases}$$

$$V(\mathbf{s}, \mathbf{g}) \leq V(\mathbf{s}, \mathbf{w}) + V(\mathbf{w}, \mathbf{g})$$

This tells us how easy it is to reach various states

This can be used for **planning** with goal conditioned value functions!

What kind of model do we need to **evaluate** a policy?

The job of the model is to **evaluate** the policy

(if you can evaluate it, you can make it better)

$$J(\pi) = E_{s \sim p(s_1)}[V^\pi(s_1)]$$

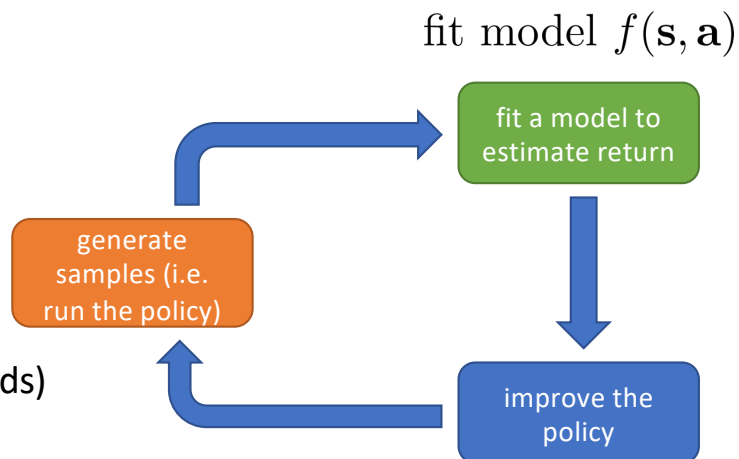
$$V^\pi(s_t) = \sum_{t'=t}^{\infty} \gamma^{t'-t} E_{p(s_{t'}|s_t)} E_{\mathbf{a}_{t'} \sim \pi(\mathbf{a}_{t'}|s_{t'})} [r(s_{t'}, \mathbf{a}_{t'})]$$

$$= \sum_{t'=t}^{\infty} \gamma^{t'-t} E_{p(s_{t'}|s_t)} [r(s_{t'})]$$

let's keep it simple
(easy to re-derive for action-dependent rewards)

$$= \sum_{t'=t}^{\infty} \gamma^{t'-t} \sum_{\mathbf{s}} p(\mathbf{s}_{t'} = \mathbf{s} | \mathbf{s}_t) r(\mathbf{s})$$

$$= \sum_{\mathbf{s}} \left(\sum_{t'=t}^{\infty} \gamma^{t'-t} p(\mathbf{s}_{t'} = \mathbf{s} | \mathbf{s}_t) \right) r(\mathbf{s})$$



What kind of model do we need to **evaluate** a policy?

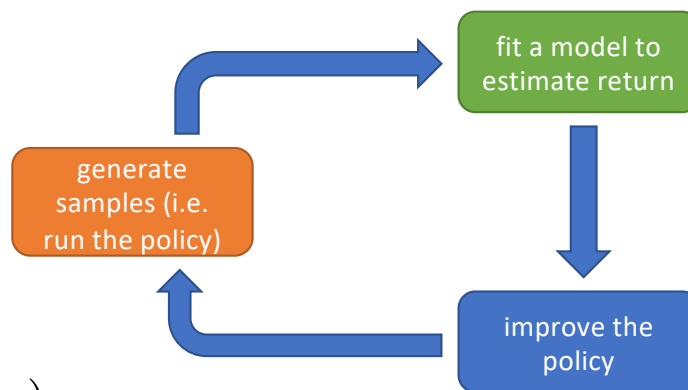
$$\begin{aligned} V^\pi(\mathbf{s}_t) &= \sum_{t'=t}^{\infty} \gamma^{t'-t} E_{p(\mathbf{s}_{t'}|\mathbf{s}_t)}[r(\mathbf{s}_{t'})] \\ &= \sum_{\mathbf{s}} \underbrace{\left(\sum_{t'=t}^{\infty} \gamma^{t'-t} p(\mathbf{s}_{t'} = \mathbf{s} | \mathbf{s}_t) \right)}_{p_\pi(\mathbf{s}_{\text{future}} = \mathbf{s} | \mathbf{s}_t)} r(\mathbf{s}) \end{aligned}$$

$$p_\pi(\mathbf{s}_{\text{future}} = \mathbf{s} | \mathbf{s}_t) = \underbrace{(1 - \gamma)}_{\text{just to ensure it sums to 1}} \sum_{t'=t}^{\infty} \gamma^{t'-t} p(\mathbf{s}_{t'} = \mathbf{s} | \mathbf{s}_t)$$

just to ensure it sums to 1

(if you can evaluate it, you can make it better)

fit model $f(\mathbf{s}, \mathbf{a})$



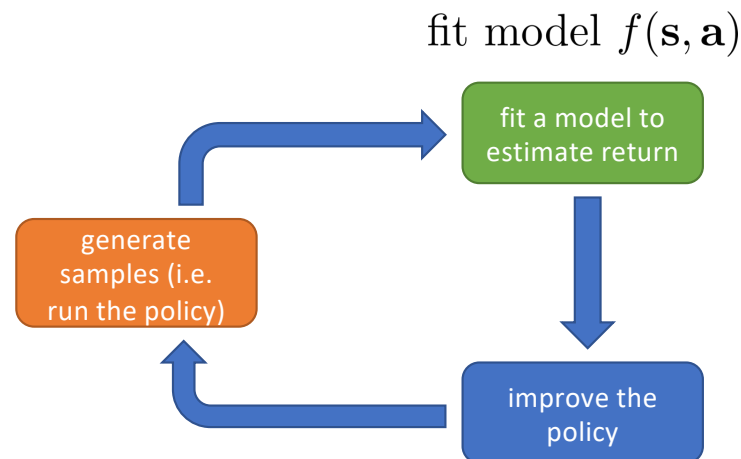
What kind of model do we need to **evaluate** a policy?

$$p_{\pi}(\mathbf{s}_{\text{future}} = \mathbf{s} | \mathbf{s}_t) = (1 - \gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p(\mathbf{s}_{t'} = \mathbf{s} | \mathbf{s}_t) \quad (\text{if you can evaluate it, you can make it better})$$

$$V^{\pi}(\mathbf{s}_t) = \frac{1}{1 - \gamma} \sum_{\mathbf{s}} \underbrace{p_{\pi}(\mathbf{s}_{\text{future}} = \mathbf{s} | \mathbf{s}_t) r(\mathbf{s})}_{\mu^{\pi}(\mathbf{s}_t)^T \vec{r}}$$

$$\mu_i^{\pi}(\mathbf{s}_t) = p_{\pi}(s_{\text{future}} = i | \mathbf{s}_t)$$

This is called a **successor representation**



Successor representations

$$\begin{aligned}\mu_i^\pi(\mathbf{s}_t) &= (1 - \gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p(\mathbf{s}_{t'} = i | \mathbf{s}_t) \\ &= \underbrace{(1 - \gamma) \delta(\mathbf{s}_t = i) + \gamma E_{\mathbf{a}_t \sim \pi(\mathbf{a}_t | \mathbf{s}_t), \mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\mu_i^\pi(\mathbf{s}_{t+1})]}_{\text{like a Bellman backup with "reward" } r(\mathbf{s}_t) = (1 - \gamma) \delta(\mathbf{s}_t = i)}\end{aligned}$$

like a Bellman backup with “reward” $r(\mathbf{s}_t) = (1 - \gamma) \delta(\mathbf{s}_t = i)$

in practice, we can use vectorized backups for all i at once

A few issues...

- Not clear if learning successor representation is easier than model-free RL
- How to scale to large state spaces?
- How to extend to continuous state spaces?

Successor features

$$\mu_i^\pi(\mathbf{s}_t) = (1 - \gamma) \sum_{t'=t}^{\infty} \gamma^{t'-t} p(\mathbf{s}_{t'} = i | \mathbf{s}_t) \quad \psi_j^\pi(\mathbf{s}_t) = \sum_{\mathbf{s}} \mu_{\mathbf{s}}^\pi(\mathbf{s}_t) \phi_j(\mathbf{s}) \quad \psi_j^\pi(\mathbf{s}_t) = \mu^\pi(\mathbf{s}_t)^T \vec{\phi}_j$$

$$V^\pi(\mathbf{s}_t) = \mu^\pi(\mathbf{s}_t)^T \vec{r}$$

$$\text{if } r(\mathbf{s}) = \sum_j \phi_j(\mathbf{s}) w_j = \phi(\mathbf{s})^T \mathbf{w}$$

$$\text{then } V^\pi(\mathbf{s}_t) = \psi^\pi(\mathbf{s}_t)^T \mathbf{w}$$

so what?

If the number of features is much less than the number of states, learning them is much easier!

$$= \sum_j \psi_j^\pi(\mathbf{s}_t) w_j$$

$$= \sum_j \mu^\pi(\mathbf{s}_t)^T \vec{\phi}_j w_j$$

$$= \mu^\pi(\mathbf{s}_t)^T \sum_j \vec{\phi}_j w_j = \mu^\pi(\mathbf{s}_t)^T \vec{r}$$

Successor features

$$\mu_i^\pi(\mathbf{s}_t) = (1 - \gamma)\delta(\mathbf{s}_t = i) + \gamma E_{\mathbf{a}_t \sim \pi(\mathbf{a}_t | \mathbf{s}_t), \mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\mu_i^\pi(\mathbf{s}_{t+1})]$$

$$\psi_j^\pi(\mathbf{s}_t) = \phi_j(\mathbf{s}_t) + \gamma E_{\mathbf{a}_t \sim \pi(\mathbf{a}_t | \mathbf{s}_t), \mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\psi_j^\pi(\mathbf{s}_{t+1})]$$

special case with
 $\phi_i(\mathbf{s}_t) = (1 - \gamma)\delta(\mathbf{s}_t = i)$



can also construct a “Q-function-like” version:

$$\psi_j^\pi(\mathbf{s}_t, \mathbf{a}_t) = \phi_j(\mathbf{s}_t) + \gamma E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t), \mathbf{a}_{t+1} \sim \pi(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})} [\psi_j^\pi(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})]$$

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx \psi^\pi(\mathbf{s}_t, \mathbf{a}_t)^T \mathbf{w} \quad \text{when } r(\mathbf{s}_t) \approx \phi(\mathbf{s}_t)^T \mathbf{w}$$

Using successor features

Idea 1: recover a Q-function very quickly

1. Train $\psi^\pi(\mathbf{s}_t, \mathbf{a}_t)$ (via Bellman backups)
2. Get some reward samples $\{\mathbf{s}_i, r_i\}$
3. Get $\mathbf{w} \leftarrow \arg \min_{\mathbf{w}} \sum_i \|\phi(\mathbf{s}_i)^T \mathbf{w} - r_i\|^2$
4. Recover $Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \approx \psi^\pi(\mathbf{s}_t, \mathbf{a}_t)^T \mathbf{w}$

Is this the **optimal** Q-function?

$$\pi'(\mathbf{s}) = \arg \max_{\mathbf{a}} \psi^\pi(\mathbf{s}, \mathbf{a})^T \mathbf{w}$$

Equivalent to **one step** of policy iteration

Better than nothing, but **not** optimal

Using successor features

Idea 2: recover many Q-functions

1. Train $\psi^{\pi_k}(\mathbf{s}_t, \mathbf{a}_t)$ for many policies π_k (via Bellman backups)
2. Get some reward samples $\{\mathbf{s}_i, r_i\}$
3. Get $\mathbf{w} \leftarrow \arg \min_{\mathbf{w}} \sum_i \|\phi(\mathbf{s}_i)^T \mathbf{w} - r_i\|^2$
4. Recover $Q^{\pi_k}(\mathbf{s}_t, \mathbf{a}_t) \approx \psi^{\pi_k}(\mathbf{s}_t, \mathbf{a}_t)^T \mathbf{w}$ for every π_k

$$\pi'(\mathbf{s}) = \arg \max_{\mathbf{a}} \max_k \psi^{\pi_k}(\mathbf{s}, \mathbf{a})^T \mathbf{w}$$

Finds the highest reward policy in **each state**

Continuous successor representations

$$\mu_i^\pi(\mathbf{s}_t) = (1 - \gamma)\delta(\mathbf{s}_t = i) + \gamma E_{\mathbf{a}_t \sim \pi(\mathbf{a}_t|\mathbf{s}_t), \mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)} [\mu_i^\pi(\mathbf{s}_{t+1})]$$



always zero for any sampled state if states are continuous

Framing successor representation as *classification*:

$$p^\pi(F = 1 | \mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}}) = \frac{p^\pi(\mathbf{s}_{\text{future}} | \mathbf{s}_t, \mathbf{a}_t)}{p^\pi(\mathbf{s}_{\text{future}} | \mathbf{s}_t, \mathbf{a}_t) + p^\pi(\mathbf{s}_{\text{future}})}$$

binary classifier

$F = 1$ means $\mathbf{s}_{\text{future}}$ is a future state from $\mathbf{s}_t, \mathbf{a}_t$ under π

$$\mathcal{D}_+ \sim p^\pi(\mathbf{s}_{\text{future}} | \mathbf{s}_t, \mathbf{a}_t) \quad \mathcal{D}_- \sim p^\pi(\mathbf{s})$$

Continuous successor representations

$$\mathcal{D}_+ \sim p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t) \quad \mathcal{D}_- \sim p^\pi(\mathbf{s})$$

$$p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}}) = \frac{p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t)}{p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t) + p^\pi(\mathbf{s}_{\text{future}})}$$

$$p^\pi(F = 0|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}}) = \frac{p^\pi(\mathbf{s}_{\text{future}})}{p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t) + p^\pi(\mathbf{s}_{\text{future}})}$$

$$\frac{p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}})}{p^\pi(F = 0|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}})} = \frac{p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t)}{p^\pi(\mathbf{s}_{\text{future}})}$$

$$\frac{p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}})}{p^\pi(F = 0|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}})} p^\pi(\mathbf{s}_{\text{future}}) = p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t)$$


 constant independent of $\mathbf{a}_t, \mathbf{s}_t$

The C-Learning algorithm

$$\mathcal{D}_+ \sim p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t) \quad \mathcal{D}_- \sim p^\pi(\mathbf{s})$$

$$p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}}) = \frac{p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}})}{p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{\text{future}}) + p^\pi(\mathbf{s}_{\text{future}})}$$

To train:

1. Sample $\mathbf{s} \sim p^\pi(\mathbf{s})$ (e.g., run policy, sample from trajectories)
2. Sample $\mathbf{s} \sim p^\pi(\mathbf{s}_{\text{future}}|\mathbf{s}_t, \mathbf{a}_t)$ (e.g., pick $\mathbf{s}_{t'}$ where $t' = t + \Delta$, $\Delta \sim \text{Geom}(\gamma)$)
3. Update $p^\pi(F = 1|\mathbf{s}_t, \mathbf{a}_t, \mathbf{s})$ using SGD with cross entropy loss

This is an **on policy** algorithm

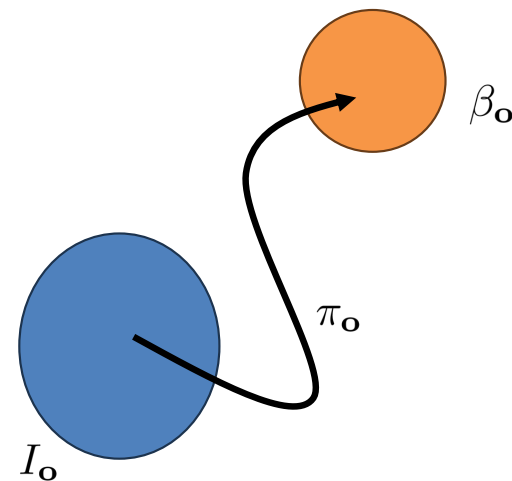
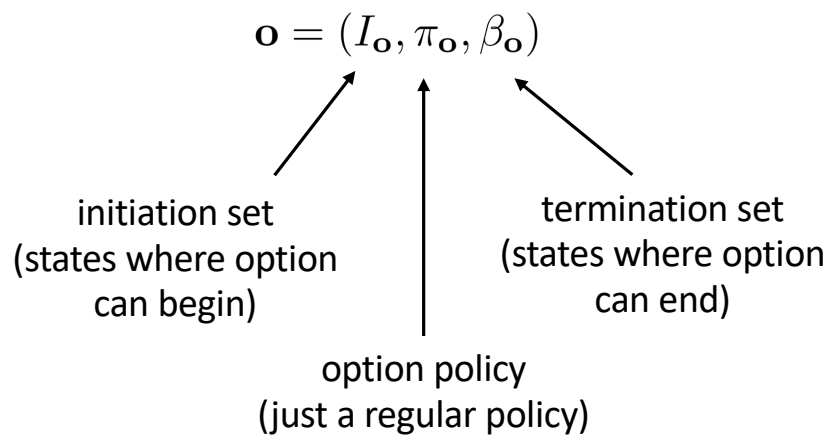
Could also derive an **off policy** algorithm

Part 5: Hierarchical RL

A classical perspective on hierarchy

options

a “mini-policy”, like a skill



choose action space $\mathcal{O} \cup \mathcal{A}$

learn $\pi_{\theta}(\mathbf{o}|\mathbf{s})$ and $Q(\mathbf{s}, \mathbf{o})$

how long option $\mathbf{o}_t$ is active

$$Q(\mathbf{s}_t, \mathbf{o}_t) \leftarrow \sum_{t'=t}^{t+h-1} r(\mathbf{s}_{t'}) + \gamma^h \max_{\mathbf{o}_{t+h}: \mathbf{s}_{t+h} \in I_{\mathbf{o}_{t+h}}} Q(\mathbf{s}_{t+h}, \mathbf{o}_{t+h})$$

Why is this a good idea?

image h is pretty big...

$$Q(\mathbf{s}_t, \mathbf{o}_t) \leftarrow \sum_{t'=t}^{t+h-1} r(\mathbf{s}_{t'}) + \gamma^h \max_{\mathbf{o}_{t+h} : \mathbf{s}_{t+h} \in I_{\mathbf{o}_{t+h}}} Q(\mathbf{s}_{t+h}, \mathbf{o}_{t+h})$$

effective horizon for the “high-level” Q-function is much lower!

smart options might also really simplify exploration

Why is this a bad idea?

$$\mathbf{o} = (I_{\mathbf{o}}, \pi_{\mathbf{o}}, \beta_{\mathbf{o}})$$

Where do these come from?

Learning options end-to-end together with the policy might negate much of the benefit!

See: Bacon et al., **The Option-Critic Architecture**

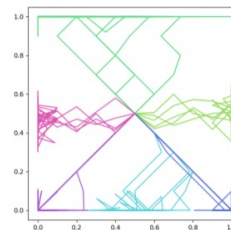
Hierarchical RL with multi-task policies

$$\mathbf{o} = (\cancel{I_{\mathbf{o}}}, \pi_{\mathbf{o}}, \cancel{\beta_{\mathbf{o}}}) \longrightarrow \pi(\mathbf{a}|\mathbf{s}, \mathbf{o}) \quad \text{or} \quad \pi(\mathbf{a}|\mathbf{s}, \mathbf{z})$$

options **are** a multi-task policy

One potential algorithm:

1. train $\pi_{\text{lo}}(\mathbf{a}|\mathbf{s}, \mathbf{z})$ using skill discovery
2. train $\pi_{\text{hi}}(\mathbf{z}|\mathbf{s})$ with regular RL
switch to a new $\mathbf{z}$ every k steps



$$I(\mathbf{z}; \mathbf{s})$$

$$r(\mathbf{s}, \mathbf{z}) = \log p(\mathbf{z}|\mathbf{s})$$

$$Q(\mathbf{s}_t, \mathbf{o}_t) \leftarrow \sum_{t'=t}^{t+h-1} r(\mathbf{s}_{t'}) + \gamma^h \max_{\mathbf{o}_{t+h} : \mathbf{s}_{t+h} \in I_{\mathbf{o}_{t+h}}} Q(\mathbf{s}_{t+h}, \mathbf{o}_{t+h})$$

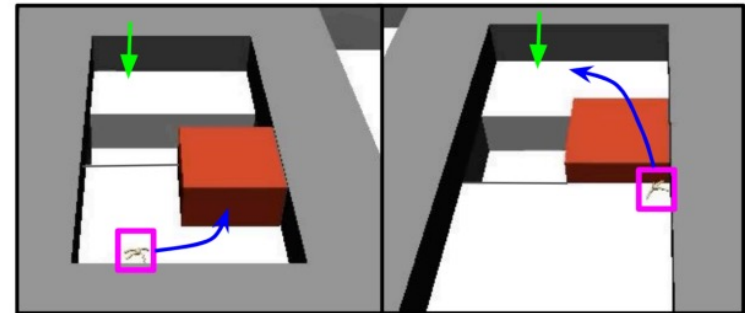
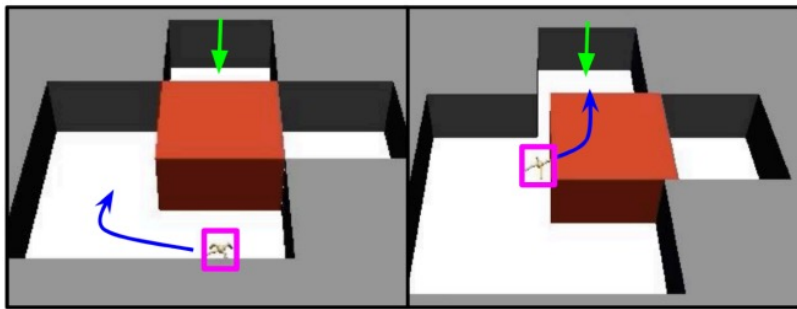
$$Q(\mathbf{s}_t, \mathbf{z}_t) \leftarrow \sum_{t'=t}^{t+k-1} r(\mathbf{s}_{t'}) + \gamma^k \max_{\mathbf{z}_{t+h}} Q(\mathbf{s}_{t+h}, \mathbf{z}_{t+h})$$

Hierarchies with goals

what if $\mathbf{z}$ is actually a goal $\mathbf{g}$?

1. train $\pi_{\text{lo}}(\mathbf{a}|\mathbf{s}, \mathbf{g})$ using any goal-conditioned RL method
2. train $\pi_{\text{hi}}(\mathbf{g}|\mathbf{s})$ with regular RL
switch to a new $\mathbf{g}$ every k steps

This works really well if we have a good goal-conditioned policy!

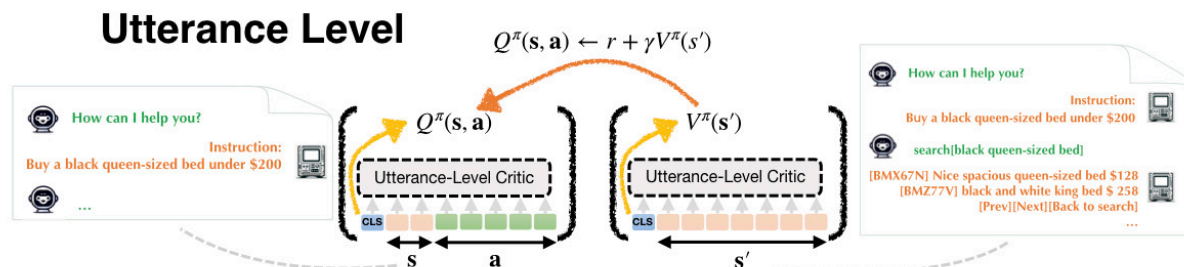


Nachum et al. Data-Efficient Hierarchical Reinforcement Learning

Hierarchies with language

what if $\mathbf{z}$ is actually a string of text ℓ ?

1. train $\pi_{\text{lo}}(\mathbf{a}|\mathbf{s}, \ell)$ to follow various language instructions
2. train $\pi_{\text{hi}}(\ell|\mathbf{s})$ with LLM RL



Zhou et al. **ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL**

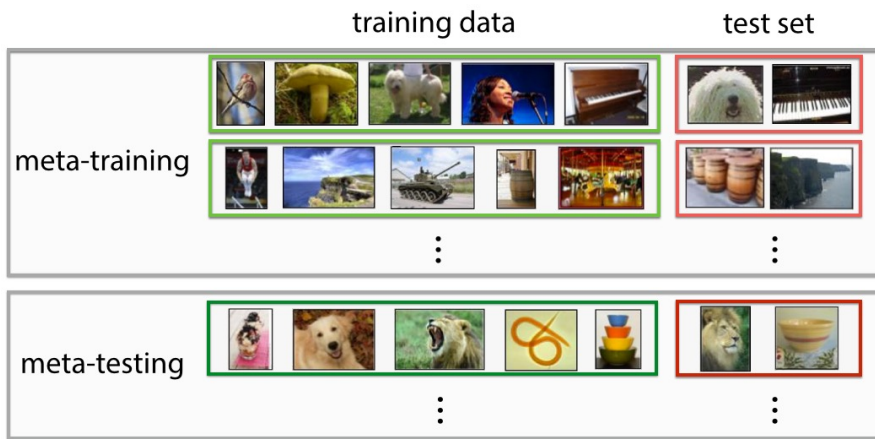
Part 6: Meta-learning

Meta-learning with supervised learning



image credit: Ravi & Larochelle '17

Meta-learning with supervised learning

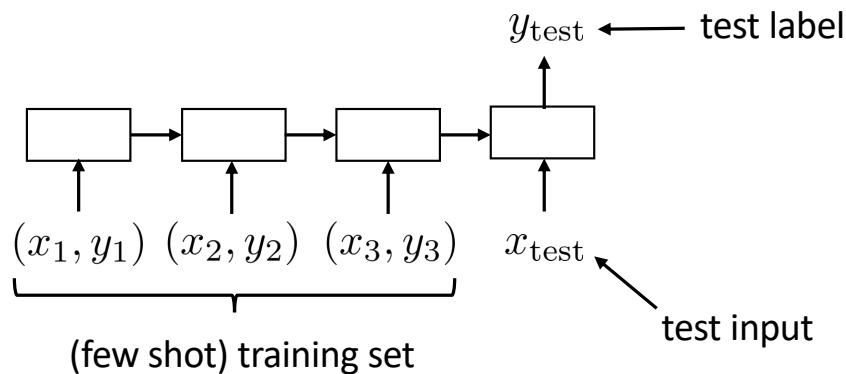


supervised learning: $f(x) \rightarrow y$

input (e.g., image) output (e.g., label)

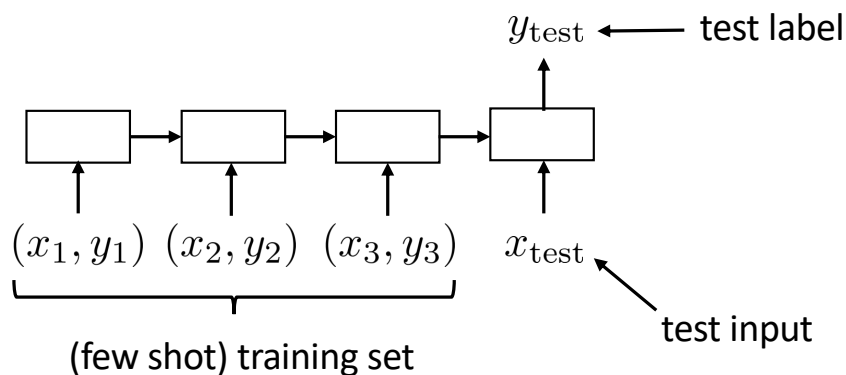
supervised meta-learning: $f(\mathcal{D}^{\text{tr}}, x) \rightarrow y$

training set



- How to read in training set?
 - Many options, RNNs can work
 - More on this later

What is being “learned”?



supervised meta-learning: $f(\mathcal{D}^{\text{tr}}, x) \rightarrow y$

“Generic” learning:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta, \mathcal{D}^{\text{tr}})$$

$$= f_{\text{learn}}(\mathcal{D}^{\text{tr}})$$

“Generic” meta-learning:

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^n \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{ts}})$$

$$\text{where } \phi_i = f_{\theta}(\mathcal{D}_i^{\text{tr}})$$

What is being “learned”?

“Generic” learning:

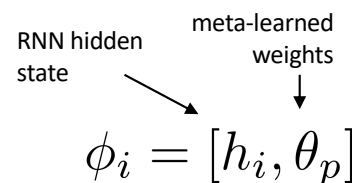
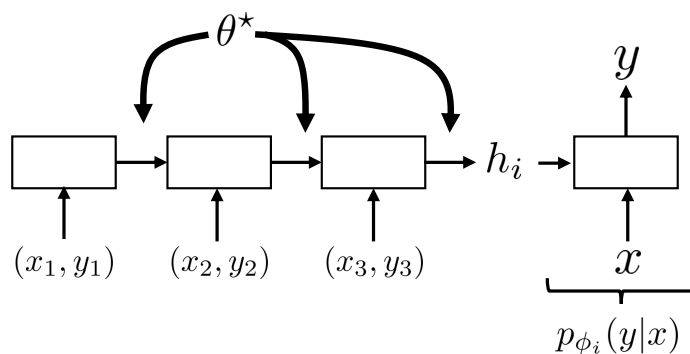
$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta, \mathcal{D}^{\text{tr}})$$

$$= f_{\text{learn}}(\mathcal{D}^{\text{tr}})$$

“Generic” meta-learning:

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^n \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{ts}})$$

$$\text{where } \phi_i = f_{\theta}(\mathcal{D}_i^{\text{tr}})$$



The meta reinforcement learning problem

“Generic” learning:

$$\begin{aligned}\theta^* &= \arg \min_{\theta} \mathcal{L}(\theta, \mathcal{D}^{\text{tr}}) \\ &= f_{\text{learn}}(\mathcal{D}^{\text{tr}})\end{aligned}$$

Reinforcement learning:

$$\begin{aligned}\theta^* &= \arg \max_{\theta} E_{\pi_{\theta}(\tau)}[R(\tau)] \\ &= f_{\text{RL}}(\mathcal{M})\end{aligned}\quad \mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, r\}$$

↖
MDP

“Generic” meta-learning:

$$\begin{aligned}\theta^* &= \arg \min_{\theta} \sum_{i=1}^n \mathcal{L}(\phi_i, \mathcal{D}_i^{\text{ts}}) \\ &\text{where } \phi_i = f_{\theta}(\mathcal{D}_i^{\text{tr}})\end{aligned}$$

Meta-reinforcement learning:

$$\begin{aligned}\theta^* &= \arg \max_{\theta} \sum_{i=1}^n E_{\pi_{\phi_i}(\tau)}[R(\tau)] \\ &\text{where } \phi_i = f_{\theta}(\mathcal{M}_i)\end{aligned}$$

↖
MDP for task i

The meta reinforcement learning problem

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n E_{\pi_{\phi_i}(\tau)}[R(\tau)]$$

where $\phi_i = f_{\theta}(\mathcal{M}_i)$

assumption: $\mathcal{M}_i \sim p(\mathcal{M})$

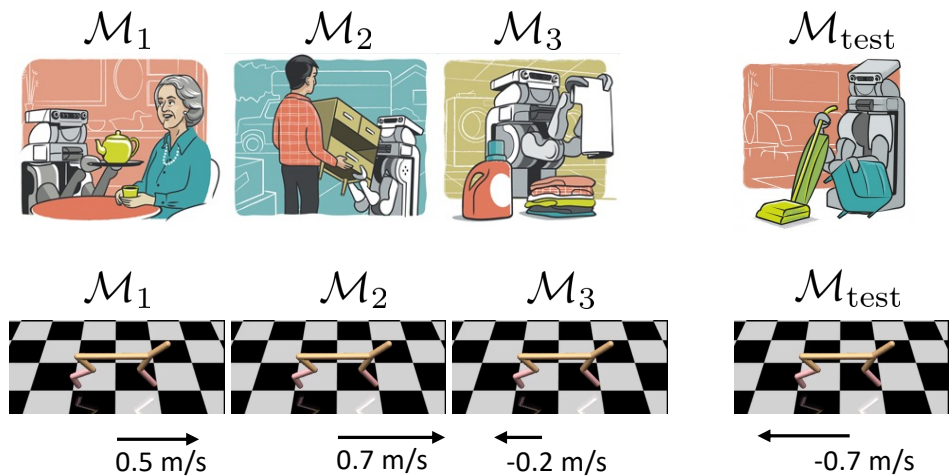
meta test-time:

sample $\mathcal{M}_{\text{test}} \sim p(\mathcal{M})$, get $\phi_i = f_{\theta}(\mathcal{M}_{\text{test}})$

$\{\mathcal{M}_1, \dots, \mathcal{M}_n\}$

↑
meta-training MDPs

Some examples:



Contextual policies and meta-learning

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n E_{\pi_{\phi_i}(\tau)}[R(\tau)]$$

where $\phi_i = f_{\theta}(\mathcal{M}_i)$



$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n E_{\pi_{\theta}}[R(\tau)]$$

$$\pi_{\theta}(a_t | s_t, \underbrace{s_1, a_1, r_1, \dots, s_{t-1}, a_{t-1}, r_{t-1}}_{\text{context}})$$

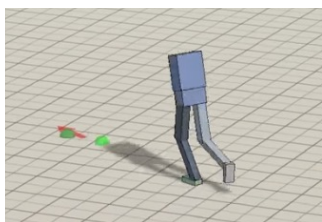
context used to infer whatever we need to solve $\mathcal{M}_i$
i.e., z_t or ϕ_i (which are really the same thing)

in meta-RL, the *context* is inferred from experience from $\mathcal{M}_i$

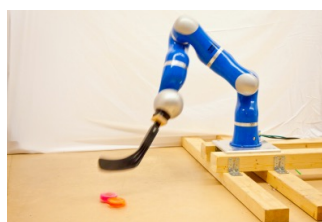
in multi-task RL, the context is typically given

$$\pi_{\theta}(a_t | s_t, \phi_i)$$

↑
“context”



ϕ : walking direction

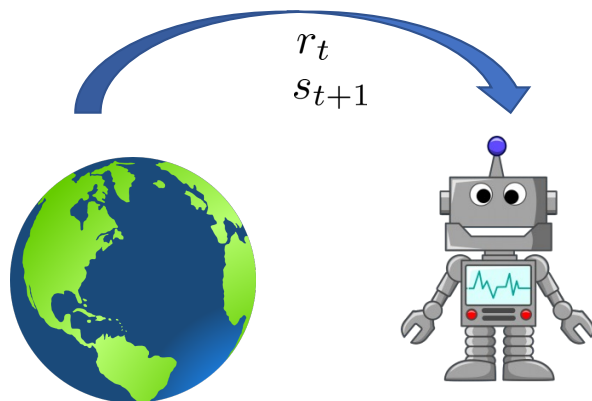


ϕ : where to hit puck

Meta-RL with recurrent policies

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n E_{\pi_{\phi_i}(\tau)}[R(\tau)]$$

where $\phi_i = f_{\theta}(\mathcal{M}_i)$

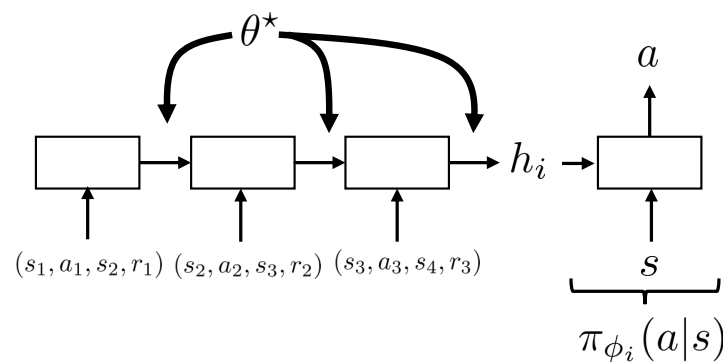


use (s_t, a_t, s_{t+1}, r_t) to improve π_{θ}

main question: how to implement $f_{\theta}(\mathcal{M}_i)$?

what should $f_{\theta}(\mathcal{M}_i)$ do?

1. improve policy with experience from $\mathcal{M}_i$
 $\{(s_1, a_1, s_2, r_1), \dots, (s_T, a_T, s_{T+1}, r_T)\}$
2. (new in RL): choose how to interact, i.e. choose a_t
 meta-RL must also *choose* how to *explore*!

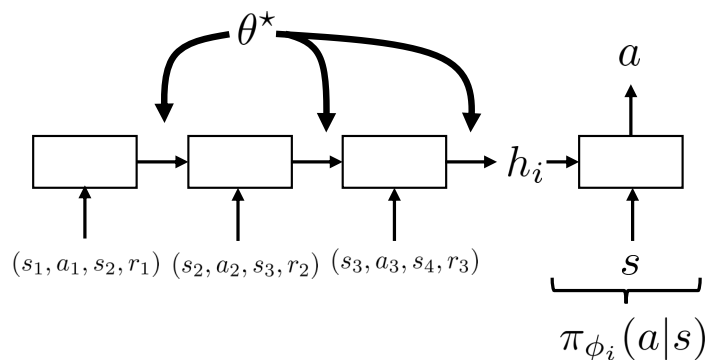


RNN hidden state
 meta-learned weights
 as before, $\phi_i = [h_i, \theta_{\pi}]$

Meta-RL with recurrent policies

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^n E_{\pi_{\phi_i}(\tau)}[R(\tau)]$$

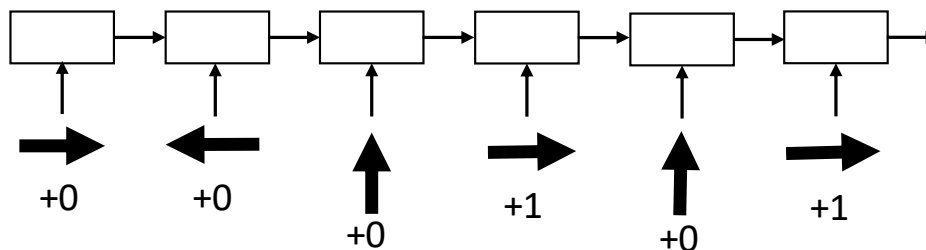
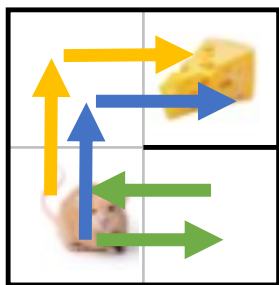
where $\phi_i = f_{\theta}(\mathcal{M}_i)$



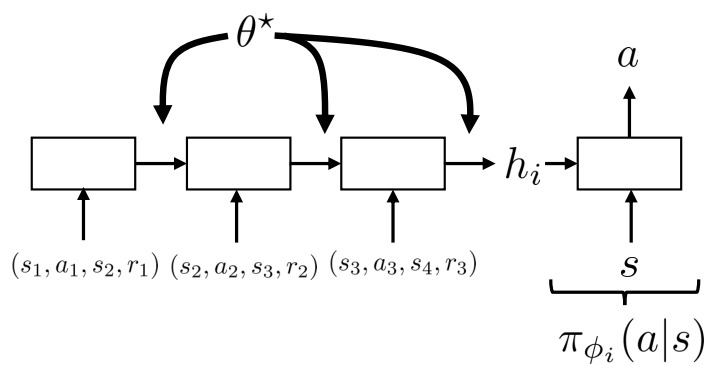
so... we just train an RNN policy?

yes!

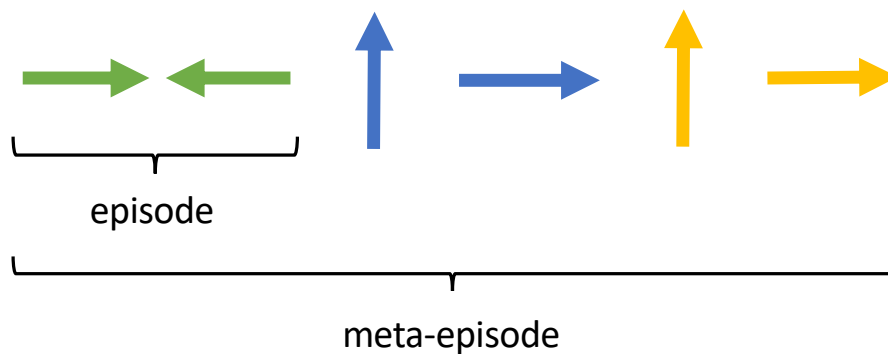
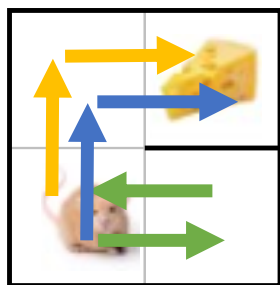
crucially, RNN hidden state is **not** reset between episodes!



Why recurrent policies *learn to explore*



1. improve policy with experience from $\mathcal{M}_i$
 $\{(s_1, a_1, s_2, r_1), \dots, (s_T, a_T, s_{T+1}, r_T)\}$
2. (new in RL): choose how to interact, i.e. choose a_t
 meta-RL must also *choose* how to *explore*!



$$\theta^* = \arg \max_{\theta} E_{\pi_{\theta}} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

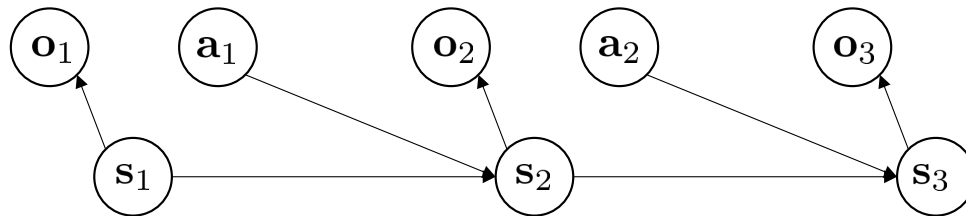
optimizing total reward over the entire **meta-episode** with RNN policy **automatically** learns to explore!

Meta-RL as... partially observed RL?

$$\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{P}, \mathcal{E}, r\}$$

$\mathcal{O}$ – observation space observations $o \in \mathcal{O}$ (discrete or continuous)

$\mathcal{E}$ – emission probability $p(o_t|s_t)$



policy must act on observations o_t !

$$\pi_{\theta}(a|o)$$

typically requires *either*:

explicit state estimation, i.e. to estimate $p(s_t|o_{1:t})$

policies with memory

Meta-RL as... partially observed RL?

$$\pi_{\theta}(a|\overbrace{s, z}^{\tilde{s}})$$

encapsulates information policy
needs to solve current task

learning a task = inferring z

from *context* $(s_1, a_1, s_2, r_1), (s_2, a_2, s_3, r_2), \dots$

this is just a POMDP!

before: $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \mathcal{P}, r\}$

now: $\tilde{\mathcal{M}} = \{\tilde{\mathcal{S}}, \mathcal{A}, \tilde{\mathcal{O}}, \tilde{\mathcal{P}}, \mathcal{E}, r\}$

$$\tilde{\mathcal{S}} = \mathcal{S} \times \mathcal{Z} \quad \tilde{s} = (s, z)$$

$$\tilde{\mathcal{O}} = \mathcal{S} \quad \tilde{o} = s$$

key idea: solving the POMDP $\tilde{\mathcal{M}}$ is equivalent to meta-learning!

typically requires *either*:

explicit state estimation, i.e. to estimate $p(s_t|o_{1:t})$

policies with memory

Policies with memory are the simplest way to do this (“in-context learning”)

There are many other ways to do this, including variational inference, gradient-based methods, etc.